

GridWise: Final Submission Documentation & Checklist

Project Name: Smart Campus Energy Optimizer **Target:** BUP CSE FEST 2026 Hackathon (Online Preliminary Round)

This document serves as the final proof of compliance against all rules, schemas, and optimization constraints defined in the canonical problem statement.

1. Compliance Checklist

[!IMPORTANT] The following checklist maps directly to the judging criteria outlined in the Problem Statement.

API & Delivery

- [x] **Deployed Public HTTP API:** API is built using FastAPI and ready for cloud deployment.
- [x] **GET /health:** Responds with `{"status": "ok"}` in under 60 seconds (typically $\sim 2\text{ms}$).
- [x] **POST /optimize-energy:** Receives the canonical JSON schema and responds with the optimized output.
- [x] **Robustness:** Handles malformed JSON safely via `RequestValidationError` (HTTP 400), returning a controlled error instead of a stack trace.

LLM Interpretation (`src/interpreter.py`)

- [x] **Required LLM Path:** Operator notes are passed to an LLM (Gemini 2.5 Flash / OpenAI / Groq) via API keys provided in `.env`.
- [x] **Supported Directives:** Outputs exclusively map to `solar_reduction`, `minimum_battery_reserve`, `no_charge_window`, `no_discharge_window`, `max_grid_window`, or `no_op`.
- [x] **Paraphrase Robustness:** Handles varying language expressions (e.g., "drops to 20%" vs "80% reduction" both translate to a factor of `0.20`).
- [x] **Applies Semantics:** `applies = false` strictly forces `no_op` and `structured_adjustment = null`.

Guardrails (`src/guardrails.py`)

- [x] **One-to-One Mapping:** Every operator note produces exactly one `directive_interpretation` entry in `note_index` order ($0..N-1$).
- [x] **Numeric Constraints Validated:** Extracted hours are unique integers sorted ascending (0 through 23). Time spans like "1 PM to 3 PM" are properly mapped to `[13, 14]` (start-inclusive, end-exclusive).
- [x] **Value Clamping:** Factors are constrained between `0.0` and `1.0`. Reserve bounds cannot exceed the battery capacity.

Mathematical Optimizer (`src/optimizer.py`)

- [x] **Energy Balance:** `grid + solar_used + battery_discharge == demand + battery_charge` is strictly enforced for every hour.
- [x] **Solar Constraints:** Unused solar is curtailed. `solar_used_kwh <= effective_solar_kwh`.
- [x] **Battery Limits:**
 - charge and discharge are mutually exclusive (implemented via binary constraints).
 - Rate bounds (`max_charge_kwh_per_hour` and `max_discharge_kwh_per_hour`) are respected.
- [x] **Battery Neutrality:** `E_after[23] == initial_energy_kwh` ensures starting energy is not consumed as a free resource.
- [x] **Cost Minimization:** Total grid cost `SUM(grid_kwh * tariff)` is strictly minimized using the PuLP / HiGHS solver.

GitHub & Documentation

- [x] **Private Repo (Pre-Deadline):** Development occurred in a private repository.
- [x] **Comprehensive README:** Contains source setup, environment variables, dependencies, Docker instructions, and curl commands.
- [x] **Docker Fallback:** Multi-platform Dockerfile included binding to 0.0.0.0 with no baked-in secrets.

2. API Contract Verification

Request Handling

The API strictly adheres to the requested input schema:

- `scenario_id` (string)
- `operator_notes` (array of strings, length 1-3)
- `hours` (array of exactly 24 hourly objects with demand, solar, tariff)
- `battery` (object with capacity, initial, minimum, and max rates)

Response Generation

The API outputs the exact requested fields:

1. `scenario_id`: Echoes the input ID.
2. `directive_interpretation`: The validated LLM-extracted rules.
3. `hourly_plan`: Array of 24 objects detailing `grid_kwh`, `solar_used_kwh`, `battery_action`, `battery_kwh`, and `battery_energy_after_kwh`.
4. `total_grid_kwh`: Accurately summed from `hourly_plan`.
5. `total_cost_bdt`: Accurately calculated `SUM(grid_kwh * tariff)`.
6. `peak_grid_kwh`: Peak single-hour grid import.
7. `plan_summary`: A human-readable text summarizing the output.

3. Technology Stack Justification

Requirement	Choice	Reason
HTTP Framework	FastAPI	High performance, native JSON schema validation via Pydantic matching the PDF exact schemas.
LLM Provider	Gemini / OpenAI	Capable of zero-shot semantic extraction. Handled securely without exposing prompts.
Optimizer	PuLP (HiGHS backend)	Exact MILP solver ensuring cost optimality and strict adherence to physical limits.
UI (Bonus)	Vanilla HTML/JS	A lightweight dashboard (served at /) for debugging and recording the 3-minute video tie-breaker, adding 0 overhead to the API.

[!TIP] **Exporting to PDF:** To generate a PDF from this document, you can right-click this file in VSCode and select "Markdown PDF: Export (pdf)" or use your browser's Print to PDF function when viewing this on GitHub.